LifelongAgentBench Database Experiment

What was set up, what was done, and what the results were

Summary

This experiment ran the database tasks from the LifelongAgentBench paper (arXiv 2505.11942v3) on a laptop, using the repository's own agent and a free local model. It tested whether showing the agent past successful attempts helps it solve new database tasks.

On the same 100 tasks, the agent solved 19 without examples and 71 with examples. Most of the gain came from the agent following the required response format, rather than from improving over time. This is one run on one quantised model, so it is a working demonstration of the effect, not a reproduction of the paper's numbers.

1. The setup

The benchmark

LifelongAgentBench gives an agent a long sequence of tasks. Only the database part was used: 500 tasks in which the agent reads a question, runs SQL against a real MySQL database, and gives a final answer. Each task starts a fresh conversation with its own tables, and each one tests some of 22 SQL skills, such as filtering, grouping and subqueries. Of the 500 tasks, 350 are graded on a direct answer and 150 are graded by comparing a hash of the final table.

Components

Part	What was used
Dataset	Public LifelongAgentBench dataset, database tasks only
Agent and evaluator	The repository's own agent and grading code, with the agent's behaviour unchanged
Database	MySQL 9.2 in a Docker container
Model	Llama 3.1 8B, 4-bit quantised (4.9 GB), served locally by Ollama
Model settings	Temperature 0, seed 0, up to 512 output tokens per reply, context window of 8,192 tokens
Hardware	Ryzen 7 7435HS, about 24 GB RAM, RTX 4050 with 6 GB VRAM; the model ran 32% on CPU and 68% on GPU
Software	Python 3.11 on Windows, with Docker Desktop managing its own Linux backend

Why a local model

The first choice of model had been retired and was no longer available. The paper's model, Llama 3.1 8B, was chosen instead. The experiment had to use free models only, so it was run locally rather than through a paid service. The paper's exact model files were not used: this is a heavily compressed 4-bit version.

Changes made to the code

The agent's logic was left alone. Four kinds of change were needed:

- **Data conversion:** a small script converted the published dataset into the format the code expects, and a separate answer file was created for the perfect-answer check described below.
- **Compatibility:** the MySQL image version was set, the database port was bound to the local machine only, and one library version was pinned.

- **Measurement:** a logging module recorded latency, prompt tokens and output tokens for every model call, linked to the task number.
- **Starting examples:** an optional setting was added so a run could begin with successful attempts saved from an earlier run. The selection rule is the same one the repository already uses.

2. What was done

Step 1: Check the system with perfect answers

Before using any model, the correct answers from the dataset were fed through the full system. All **500 of 500** were graded correct, which showed that the tasks, database and grading worked together. One early attempt was cut off by a time limit I had set, so it was rerun in full.

Step 2: Short trial with Llama

A 5-task trial confirmed the agent, model and logging worked together. Llama solved 1 of 5. Two tasks failed on response format and two ran out of turns.

Step 3: The main experiment

Three runs were carried out in order:

Run	Tasks	Examples shown	Purpose	Duration
1	0-99	None	Collect successful attempts	43 min
2	100-199	None	Baseline for comparison	39 min
3	100-199	4 most recent successes	Measure the effect of examples	About 33 min

Run 3 began with the 4 most recent successful attempts from Run 1. After that, each new success replaced the oldest example, so the 4 examples kept changing. The examples were pasted into the agent's first prompt. Only attempts that were both correct and properly finished could be used.

Runs 2 and 3 used identical tasks in the same order, and the examples were the only intended difference. This allows a direct task-by-task comparison. Run 1 is not a fair baseline for Run 3 because it used different tasks.

Problems during the experiment

- Run 3 crashed 2 seconds after its first start. I had added the new starting-examples setting without giving it the default value the configuration system requires, and my earlier test had skipped that check. Runs 1 and 2 were unaffected. After the fix, only Run 3 was started again, and no task results were lost.
- After the restart, I checked that the examples really reached the model: the first prompt in Run 3 was 2,708 tokens, against about 600 without examples, and the prompts contained past worked solutions.

3. Results

Overall

Measure	Run 1: no examples	Run 2: no examples	Run 3: with examples
Tasks	0-99	100-199	100-199
Correct	**29**	**19**	**71**
Finished with an answer	39	31	88
Ran out of turns	36	46	10
Rejected for bad format	25	23	2
Correct as a share of finished tasks	74% (29/39)	61% (19/31)	81% (71/88)
Model calls per task	2.71	2.88	2.25
Average prompt tokens per call	851	822	2,067
Highest prompt tokens	2,386	1,683	3,117
Average output tokens per call	128	114	97
Average seconds per call	8.0	6.7	7.2

"Finished with an answer" means only that the agent gave a final answer in time, and that answer could still be wrong. The slowest call in Run 1 took 110 seconds because the model was being loaded for the first time.

Task-by-task comparison of Runs 2 and 3

Outcome across the same 100 tasks	Tasks
Only Run 3 was correct	56
Only Run 2 was correct	4
Both runs gave the same result	40

An exact sign test on the 60 tasks where the runs disagreed gives $p < 0.001$, so chance is a very unlikely explanation.

Results by SQL skill

Each task tests several skills, so the task counts below add up to more than 100. Run 3 scored at least as well as Run 2 on all 22 skills and scored higher on 21.

Skill	Tasks	Run 2 correct	Run 3 correct
column_alias	35	8	26
delete	14	2	8
group_by_multiple_columns	14	3	12
group_by_single_column	17	5	10
having_aggregate_calculation	4	1	2
having_multiple_conditions_with_aggregate	9	3	6
having_single_condition_with_aggregate	16	4	10
insert	9	0	6
limit_and_offset	6	0	5

Skill	Tasks	Run 2 correct	Run 3 correct
limit_only	15	4	11
order_by_multiple_columns_different_directions	13	3	9
order_by_multiple_columns_same_direction	8	4	4
order_by_single_column	7	3	6
select	85	19	57
subquery_multiple	17	0	11
subquery_nested	20	3	8
subquery_single	9	1	8
table_alias	25	4	14
update	9	0	7
where_multiple_conditions	24	3	19
where_nested_conditions	10	1	5
where_single_condition	29	5	22

Cost

Examples make every prompt longer. Total prompt tokens rose from about 237,000 in Run 2 to about 465,000 in Run 3. Even so, total time spent waiting on the model fell from about 32 to 27 minutes, because Run 3 needed fewer calls (225 against 288).

4. What the results mean

- **Most of the gain is about format.** Format rejections and running out of turns together fell from 69 tasks to 12. Without examples, Llama often did not respond in the way the system requires.
- **Answers also improved.** Correct answers rose from 61% to 81% of finished tasks.
- **There was no sign of improvement over time.** Run 3 solved 34 of its first 50 tasks and 37 of its last 50. The gain was there from the start, which looks like few-shot prompting rather than gradual learning.
- **The model itself did not change.** Its weights stay fixed throughout. Only the examples in the prompt change, and this experiment did not measure forgetting.
- **Task order is a weak source of learning here.** A separate check of the 500-task sequence found that neighbouring tasks were no more related than random pairs (0.302 against 0.305). All 22 skills had already appeared by task 26.

5. Limitations

- Each condition was run once, with one model and one setting of 4 examples.
- Only 200 of the 500 tasks were used, and the examples condition covered only tasks 100-199.
- Tasks 100-199 appear harder than tasks 0-99 (19 against 29 correct with no examples).
- The model was a 4-bit version, and the MySQL version was my best guess because the exact version was not confirmed. Scores should not be compared directly with the paper.

- The paper's difficulty-level results were not reproduced because every task in this data had difficulty level 0.
- The repository had no ready settings for 2 or 8 examples, and these were not run. Larger settings such as 16 or more would need a bigger context window, and they would be slower on this hardware.
- Prompts never came near the 8,192-token limit, so no text was cut off in these runs.
- The grading code was used as supplied and passed the perfect-answer check, but I only skimmed how direct answers are compared and did not audit it line by line.